If the user hasn't been persisted, it means that the `first_or_create` method, which was called, failed in saving the user in the database. Therefore, we redirect our user into the new user registration form to correct any mistakes that may occur during this automated process. However, we must store the attributes somewhere to make sure that the form is preloaded with authentication information, which we got from Twitter; so, let's store it in the session with the `devise.user_attributes` key.

We must tell Devise to automatically preload forms with attributes that were stored in the session if these attributes are available. To do this, we can override Devise's `new_with_session` class method in our `User` model. This override checks whether the session information with the `devise.user_attributes` key exists. If it does, it will assign the attributes automatically. It will also assign attributes from `params` if it's available, and run the validation so that error messages will instantly pop up when the user opens the form.

```
def self.new_with_session(params, session)
  if session["devise.user_attributes"]
    new(session["devise.user_attributes"], without_protection: true)
do |user|
      user.attributes = params
      user.valid?
    end
  else
    super
  end
end
```

At this point, we will actually have a fully working authentication system, which uses remote authentication to verify user credentials. However, our application still has problems that need to be ironed out. Devise will still prompt that users have to supply a password when submitting their registration form, although it isn't necessary anymore as now they can use remote authentication. To change this behavior, we can override Devise's `password_required?` method so that Devise can skip password validation if the `provider` field isn't blank:

```
def password_required?
  super && provider.blank?
end
```


www.it-ebooks.info